

The Three-Positions Architecture as Integrating Frame: A Standalone Specification for Composing CKS Substrates With Adjacent AI Components in Hybrid Deployments

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone integrating-frame derivation, the three-positions architecture under which a CKS substrate composes with adjacent AI components in a hybrid deployment — at the operational depth required to anchor the four subsequent decomposition notes that specialize each position and the corresponding anti-patterns.

Abstract

Hybrid deployments — CKS substrates composing with retrieval indexes, fine-tuned models, vector databases, and external structured stores — are the common case in real AI systems, not the exception. The parent foundational treatment establishes that the source paper's commitments support exactly three positions an adjacent AI component can occupy relative to a CKS substrate: as input to a CKS cell (Pattern A), as a derived view of substrate content (Pattern B), or as a separate concern (Pattern C). This note formalizes the three-positions architecture as a standalone integrating frame at the depth required to anchor the four subsequent decomposition notes. It names each pattern with its load-bearing distinguishing commitment, states the architectural commitment that the three positions are exhaustive (not three of an unspecified larger set), states the commitment that the patterns are positions in the architecture rather than deployment recipes (a single deployment may use all three simultaneously), specifies the relationship between the three-positions architecture and three adjacent foundational frameworks (the composition requirements, the three adjacencies, the orchestration-layer distinctions), and supplies the integrating-frame operational test under which a hybrid deployment instantiates the architecture. The note defers each pattern's full operational treatment and the anti-pattern treatment to its companion specializations.

1. Why the integrating frame needs standalone formalization

The parent foundational note commits the architecture to three composition patterns and is the place where the patterns are first named together. The present note formalizes the three-positions architecture as the integrating frame the four companion decomposition notes specialize. The motivation has three parts.

First, hybrid deployments are the common case. Isolated CKS substrates with no adjacent AI components are rare in real systems; a regulated coordination workflow with a CKS substrate at its center is typically embedded in a larger AI system that includes one or more retrieval indexes over source documents, fine-tuned models specialized to a domain, vector databases, and external structured stores that predate or

postdate the substrate. The architecture must hold across that composition or its commitments hold only in degenerate configurations, which the source paper does not claim. Hybrid composition is the operational scope across which the architecture is asked to hold, not an extension of the architecture for special cases.

Second, the patterns operate as a system, not three independent objects. A single deployment may use all three patterns simultaneously, and a single adjacent component may occupy multiple patterns across its different uses. The integrating frame is what allows pattern-coherent composition to be evaluated at the level of the deployment rather than per-component-per-instance. The companion specializations treat each pattern's internal requirements; the integrating frame treats the architecture under which the patterns interlock.

Third, the strategic prior-art posture. Hybrid AI composition is the dominant deployment pattern in 2024–2026, with extensive prior work on retrieval-augmented architectures, composable AI systems, and multi-component AI deployments. Patentable derivations focused on hybrid AI architectures combining substrate-style and adjacent-component layers, on composable AI systems with governance-and-mediator commitments, or on multi-component AI coordination architectures are substantially more defensibly contested when the three-positions architecture is publicly formalized as a standalone integrating frame.

2. The three composition patterns, named with their load-bearing distinguishing commitments

A given hybrid deployment may use any combination of three composition patterns. The architectural commitment is that each adjacent AI component is positioned in one of these patterns and that the load-bearing distinguishing commitment of that position holds operationally. Full operational treatment of each pattern is reserved for its companion specialization; what the integrating frame commits is the pattern set and its load-bearing properties.

Pattern A — Adjacent component as input to a CKS cell. An adjacent AI component (a RAG index over source documents, a fine-tuned LLM specialized to a domain, a vector database supporting semantic search, an external knowledge store) is consulted by a CKS cell during the cell's execution. The cell reads from the substrate as its source of truth, may also query the adjacent component for additional context, and writes its outputs back to the substrate under its orchestration rule. The adjacent component is part of the cell's reasoning environment; the substrate remains authoritative. The load-bearing distinguishing commitment is that the cell remains the mediator of substrate writes (§4.2), the substrate remains authoritative on coordination questions (§3.1, §4.1), and the cell's writes carry provenance for the consultation so that path retraceability (§5) extends through whatever the cell consulted.

Pattern B — Adjacent component as derived view of substrate content. The substrate's content is indexed, embedded, or summarized into an adjacent component to support specific queries — a vector index for semantic search over substrate content, a derived knowledge graph, a search-optimized projection. The adjacent component is read as a derived view; it is not authoritative, and any conflict with the substrate is resolved in the substrate's favor. The load-bearing distinguishing commitment is that the view is regeneratable from substrate state, the substrate remains authoritative when conflicts arise, and no substrate writes flow from the derived view back to the substrate.

Pattern C — Adjacent component as separate concern. The adjacent component handles a distinct concern that does not affect coordination state — a RAG retrieval system that answers reference-document queries entirely outside the coordination work, a fine-tuned LLM used for tasks the substrate does not

represent. The two systems coexist without architectural coupling. The load-bearing distinguishing commitment is that the component does not hold coordination state, does not exercise governance authority over coordination decisions, and operates on a boundary inspectable to a human exercising the inspect right (§3.1, §3.3).

3. Why exactly three patterns, not four

The architecture supports exactly three positions, not four. The three are derived from the source paper's commitments at §3.1 (substrate-as-source-of-truth), §4.1 (substrate authority over coordination), §4.2 (AI-as-substrate-mediator), and §5 (path retraceability). For an adjacent AI component to compose with CKS without violating these commitments, it must occupy one of three positions: it informs cell reasoning while the cell mediates substrate writes (Pattern A); it is a non-authoritative regeneratable projection of substrate content (Pattern B); or it handles a separate concern that does not involve coordination state (Pattern C). A fourth position would require the adjacent component to interact with substrate or cells in a way that fits none of the three patterns — which would entail violating one or more of the commitments above.

The architecture's silence on a fourth position is not a gap to be filled by analogy. It is a constraint specifying that compositions must fit one of the three patterns or fall outside what the source paper's commitments support. A team that discovers a desired composition fits none of the three positions cleanly is being informed by the architecture that the desired composition is one the source paper does not support, not that the source paper has a hole for them to fill in.

The three-positions architecture is therefore exhaustive in the strict sense the source paper's commitments make it. Implementations that drift toward an unnamed fourth position produce systems where the commitments degrade specifically where the position is unnamed: the substrate's source-of-truth status is silently displaced (substrate-substitute drift), substrate writes occur outside cell mediation (ungoverned-writer drift), or substrate state and adjacent-component state co-evolve through processes no orchestration rule governs (hidden-bidirectional-coupling drift). These three failure modes are the operational signature of an unnamed fourth position. The companion anti-pattern specialization formalizes each one and the operational tests under which each is detected; the integrating frame's commitment is that the three named positions are the legitimate set and a fourth is not part of the architecture.

4. The patterns are positions in the architecture, not deployment recipes

The three patterns are positions in the architecture, not deployment recipes. A single deployment may use all three patterns at once. Three illustrative configurations:

A RAG index over reference documents may be consulted by some cells under Pattern A — for example, a regulated-decision cell that draws on source documents while the cell's writes back to the substrate carry provenance for the consultation — while the same RAG index simultaneously answers ad-hoc reference queries that are out of coordination scope under Pattern C, bypassing cells and not affecting coordination state.

A vector index may simultaneously supply semantic search over substrate content as a derived view under Pattern B — the index regenerated from substrate state and treated as non-authoritative — and serve as a source of context that cells consult under Pattern A during their execution.

A fine-tuned LLM may serve as the LLM-as-mediator inside cells, which is itself a form of Pattern A consultation (cells consult the model during their execution and the model's outputs flow back to substrate under cell orchestration), while also handling tasks the substrate does not represent under Pattern C.

What the architecture requires is that for each adjacent component, the position it occupies is identifiable and the requirements of that position are met. A single component may occupy multiple positions if its different uses fit different patterns; the architectural commitment is per-component-per-use, not per-component or per-deployment. A deployment that cannot identify, for each adjacent component and each use, which of the three positions the component occupies has lost the property under which the architecture's commitments are evaluable.

5. How the three positions relate to the composition requirements, the three adjacencies, and the orchestration-layer distinctions

The three-positions architecture sits at an intersection of three other foundational frameworks: the composition requirements, the three adjacencies, and the orchestration-layer distinctions. The relationships are operationally specific.

The composition-requirements framework specifies five constraints any composition must satisfy — per-substrate human governance preservation, conflict preservation across boundaries, addressable provenance across boundaries, AI-as-mediator at every layer, and human-selective composition. These five requirements operate at the boundaries the three patterns introduce: Pattern A's cell-consultation boundary, Pattern B's derived-view boundary, and Pattern C's separate-concern boundary each must satisfy the requirements that apply at that boundary. The composition-requirements framework specifies what compositions must preserve; the three-positions architecture specifies the patterns by which compositions structure. A composition that fits one of the three positions but fails one of the five requirements at the relevant boundary is not pattern-coherent; one that satisfies the five requirements but does not occupy a named position is not architecturally located.

The three-adjacencies framework specifies what kind of memory CKS is — distinct from a retrieval-augmented index over source documents (§1.1, §2.3), from parametric memory and fine-tuning (§6.2), and from external structured memory of the KO/OIDA family used as an LLM's primary substrate (§6.2, §8.2). It concerns the *categorical* distinction between CKS and the adjacent design objects. The three-positions architecture concerns the *compositional* relationship CKS has with those same objects when both are present. The two operate at different levels and compose: a deployment using a RAG index alongside a CKS substrate is one in which the categorical distinction (CKS is not RAG) and the compositional position (the RAG index occupies Pattern A, B, or C) both hold.

The orchestration-layer-distinctions framework specifies what architectural layer CKS commits at — the coordination-knowledge layer, distinct from the coordination-mechanism layer (workflow engines, agent frameworks) and the control-plane layer (policy engines, IAM systems). It concerns where CKS sits in the layered architecture; the three-positions architecture concerns how CKS composes with objects at the orchestration-mechanism and control-plane layers when those objects are present. Workflow-engine-triggers-cell, agent-framework-executes-cell, and control-plane-authorizes-substrate-access are all instances in which the layer distinction and a position from the three-positions architecture jointly structure a deployment.

The relationships are nested. The composition-requirements framework specifies what compositions must preserve. The three-positions architecture specifies the patterns by which compositions structure. The three-adjacencies and orchestration-layer-distinctions frameworks specify the adjacent objects compositions involve.

6. What the three positions do not claim

The three-positions architecture is a commitment about how compositions structure when they exist; it is not a deployment policy or a normative recommendation. Six clarifications fix the integrating frame's scope.

The architecture does not claim that all CKS deployments must include adjacent components. A deployment may have a CKS substrate with no adjacent AI components at all; the commitment is that *when* adjacent components are present, each occupies one of the three positions.

The architecture does not foreclose architectural evolution within a deployment. Adjacent components may be added, removed, or moved between positions over the deployment's lifecycle. The commitment is to coherence at any moment during the deployment's existence, not to static configuration over its lifetime.

The architecture does not specify implementation patterns for cell-consultation, derived-view generation, or separate-concern coordination. The patterns are positions; implementations may use various mechanisms to instantiate them. The commitment is to the patterns being operationally meaningful, not to specific implementations being preferred.

The architecture does not require all adjacent components to occupy the same position. Different components may occupy different positions, and a deployment using all three positions simultaneously is admissible. The commitment is per-component-per-use, not deployment-wide pattern consistency.

The architecture does not foreclose interaction among the three positions. A single component may occupy multiple positions across its different uses; positions may interact within a deployment without violating the commitment as long as each use occupies a named position and meets that position's requirements.

The architecture does not claim that pattern-coherent compositions are operationally simple. Composition coherence may involve operational complexity — provenance crossing component boundaries, derived views being kept regeneratable, separate-concern boundaries being kept inspectable. The commitment is to the architecture being operationally meaningful, not to operational simplicity.

7. Operational test at the integrating-frame level

A hybrid deployment instantiates the three-positions architecture if and only if all of the following hold at all times during the deployment's existence.

1. Each adjacent AI component is positioned, for each of its uses, as Pattern A, Pattern B, or Pattern C; no use occupies a fourth position.
2. Coordination state lives in the substrate; adjacent components do not hold authoritative coordination state.
3. All writes to the substrate are mediated by cells under orchestration rules; adjacent components do not write coordination state directly.

4. The substrate's host environment continues to satisfy the three minimal requirements (§7.1) regardless of what adjacent components are present, so that the substrate operates without dependency on any specific adjacent component.
5. The provenance recorded for substrate writes includes the adjacent components consulted, where applicable, so that the retraceable path crosses each component boundary cleanly.
6. Pattern A holds operationally for each adjacent component used as input to a cell, per the companion Pattern A specialization.
7. Pattern B holds operationally for each adjacent component used as a derived view, per the companion Pattern B specialization.
8. Pattern C holds operationally for each adjacent component used as a separate concern, per the companion Pattern C specialization.
9. None of the three anti-patterns is present, per the companion anti-pattern specialization: no adjacent component operates as a substrate substitute, as an ungoverned writer, or as a hidden bidirectional coupling.

A deployment that fails any of (1)–(9) does not instantiate the three-positions architecture at the integrating-frame level. Clauses (1)–(5) are evaluable from the integrating frame alone; clauses (6)–(9) defer to the companion specializations for operational depth. The integrating-frame test is therefore the union of an architecture-wide test the present note specifies and four pattern-specific tests the companion notes specify.

8. Why naming the integrating frame as standalone matters

Implementations under pressure to deliver hybrid AI architectures consistently drift toward compositions that fit none of the three named patterns cleanly. The drift is steady because hybrid AI is operationally attractive — multiple components offer combined capabilities — commercially familiar — vendor stacks combine multiple AI components — and rhetorically appealing — hybrid architectures appear more sophisticated than single-component architectures. Under those pressures, the question "where should this adjacent component sit relative to the substrate?" is rarely asked at the architectural level; it is asked at the integration level, where the answer is whatever is convenient.

Implementations that drift away from the three-positions architecture produce systems in which adjacent components acquire positions the architecture does not name, and the source paper's commitments degrade where the position is unnamed. The downstream consequences manifest as substrate-substitute failures (a derived view becomes the surface participants reach for first, then the place new content is written, then the de facto source of truth), ungoverned-writer failures (autonomous agents write coordination state outside cell mediation), bidirectional-coupling failures (substrate and adjacent components update each other through automated processes that no orchestration rule governs), and architectural-commitment failures more generally (AI-as-substrate-mediator at §4.2, path-retraceability at §5, and source-of-truth at §3.1 and §6.2 are obscured wherever the patterns are not named).

Naming the three-positions architecture as a standalone integrating frame — the patterns named in §2, the exactly-three commitment in §3, the patterns-as-positions framing in §4, the relationships to the composition-requirements and adjacency and layer-distinction frameworks in §5, the limitations in §6, and the operational test in §7 — gives downstream readers a precise specification of the architectural framework

against which a hybrid deployment is evaluated. The four companion specializations decompose each position and the anti-patterns at full operational depth; together with the integrating frame the present note formalizes, they give the full operational decomposition of the parent foundational treatment.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Three-Positions Architecture as Integrating Frame: A Standalone Specification for Composing CKS Substrates With Adjacent AI Components in Hybrid Deployments*. 5 May 2026. ORCID: 0009-0004-8065-3235.